CAPSTONE PROJECT: FINAL REPORT

IndieGuard: Indie Game Launch Analytics

Predicting negative-review risk and identifying winning feature combinations on Steam

Course	23CSE452 Business Analytics (Units 1, 2 and 3)
Programme / section	[programme and section]
Team number	Group 1
Guide	Rohini S
Academic year	2026-2027
Submission date	[submission date]
Repository and board	github.com/Shan713/IndieGuard; github.com/users/Shan713/projects/1
Live dashboard	https://indieguard.streamlit.app

Team member	Register number
Thotakura Srihitha	CB.SC.U4CSE23450
Gayas Sheik	CB.SC.U4CSE23118
Shantharam P	CB.SC.U4CSE23146
Pradeep Sasidharan	CB.SC.U4CSE23232
Valikala Tejaswini	CB.SC.U4CSE23752

Declaration and acknowledgement

We declare that this report and the project it describes are our own work as a team, with AI assistance disclosed in Section 8.3. The data were collected by our own scrapers from public Steam sources; no pre-built dataset was used. Prior studies, algorithms and software are cited in the reference list. Our use of generative-AI tools, what each tool supported, the code and content it affected, and how we verified and corrected it are disclosed in full in Section 8.3. Tools assisted the work; the project's research design, data collection runs, analytical decisions and final conclusions were made, run and checked by the team.

We thank our guide, Rohini S, for guidance and feedback during the capstone, and our course faculty for the structure and rubric that shaped the work.

Team member	**Signature**	**Date**
Thotakura Srihitha		
Gayas Sheik		
Shantharam P		
Pradeep Sasidharan		
Valikala Tejaswini		

Abstract

A small indie studio has little room for a bad launch. Steam lets a player refund a game bought in the last 14 days if they have played it for under two hours, and early impressions shape a game's rating. Refunds are not public, so we do not measure them; instead we study negative reviews written in that first two-hour window as a proxy for launch risk. IndieGuard asks what drives negative reviews, which feature combinations go with strong reception, and whether patches help. We collected our own data with throttled, resumable scrapers on 1,861 indie roguelike and roguelite games (2022 to 2025), 1.65 million reviews and 29,918 patch and sale events, cleaned them under 32 logged rules and split them by game. Exploratory analysis (Unit 1), five model families at review and game level with grid-search tuning (Unit 2), and two Unit 3 methods, text mining and association rule mining, were evaluated on held-out games. Reviews written inside the refund window are 21.2% negative against 8.8% later. A text model detects negative reviews with a PR-AUC of 0.84 and ROC-AUC of 0.97, but store metadata predicts a game's success tier only modestly (macro-F1 about 0.42 to 0.44 against 0.22 for guessing). Ninety-five tag combinations pass false-discovery control and about two-thirds of the testable ones hold on unseen games; patches show no clear effect against matched controls. A deployed Streamlit dashboard integrates the results and a launch-risk scorer. All findings are associations in observational data.

Executive summary

Small studios have to decide what to fix before launch, which feature combinations to back and when a patch is worth the effort, but public information gives them little to go on. We focus on negative reviews written in a game's first two hours, the period in which Steam still allows a refund. Refund numbers are not public and we do not measure them; an early negative review is used as a proxy, a sign that the first hour did not work for that player.

To study this we built our own dataset from Steam (1,861 indie roguelike games and 1.65 million reviews) and checked each result on games that were held back from the models. The main findings are:

- **The first two hours matter.** 21.2% of reviews written under two hours of play are negative, against 8.8% later; they make up 29% of all negative reviews. For games priced $20 or more the early figure is 31.8%.
- **Complaints change.** Early negative reviews lean to crashes, multiplayer bugs and controls; later ones to boss design, balance and repetition.
- **Store data alone say little.** A model that sees only store information finds a game's success tier about twice as well as guessing, which is useful for comparing launch plans but not a forecast.
- **Reading the review text works.** A text model flags negative reviews far better than store data (ROC-AUC 0.97 against 0.76 on the same reviews). It detects complaints; it does not forecast them.
- **Focused 2D, pixel, retro and procedural combinations** reach the Strong tier about two-thirds of the time (35% overall); no combination passed our thresholds for struggling launches.
- **Patches do not lift ratings by themselves.** Against matched games with no patch, the change is +0.27 points (interval -0.82 to +1.36).

From these results we suggest five actions for a studio to test, in priority order: (1) treat the first hour as a standalone product and blind-test it; (2) match the price to what the first hour delivers; (3) try a focused scope; (4) complete the store page; (5) patch for specific problems rather than to lift sentiment in general. In the illustrative scenario in Section 6.3, a first-hour fix that brings early negativity down to the level of cheap games would avoid about 14 negative reviews per 1,000; this is an estimate under stated assumptions, not a measured result. The live dashboard (https://indieguard.streamlit.app) lets a studio explore the market, the tag rules, the complaint themes, the patch results and score a planned game.

Table of contents

Declaration and acknowledgement ii

Abstract iii

Executive summary iv

Table of contents v

List of figures vii

List of tables vii

Abbreviations viii

1. Introduction, Business Objectives and Literature Review 1

    1.1 Context and scope 1

    1.2 Related analytics work and research gap 2

2. Dataset Collection, Validation and Preparation 2

    2.1 Provenance, coverage and challenges 2

    2.2 Data quality, privacy and ethics 3

    2.3 Transformations and usable dataset 4

3. Exploratory Data Analysis and Visualization (Unit 1) 5

4. Feature Engineering and Predictive Modelling (Unit 2) 6

5. Advanced Analytics (Unit 3): Two Selected Methods 8

    5.1 Method 1: Text mining 9

    5.2 Method 2: Association rule mining 11

6. Comparative Analysis and Business Impact 12

    6.1 Comparison of the two Unit 3 methods 12

    6.2 Comparison with existing studies 13

    6.3 KPI-based business impact and scenarios 13

7. Interactive Application, Figure Index and Reproducibility 14

    7.1 Mandatory dashboard 14

7.2 Figure index and reproduction map 15
7.3 Code and environment 16
8. End-to-End Validation, Traceability and Contributions 16
8.1 Completion and validity 16
8.2 Project tracking and individual work 17
8.3 AI use and originality declarations 17
9. Conclusion, Recommendations and Roadmap 18
References 20
Appendix A. Extended data dictionary and validation records 22
Appendix B. Tuning, additional figures and supplementary analysis 23
Appendix C. Implementation details and dashboard user guide 24
Appendix D. Tracking-board evidence, contribution records and AI declaration 25

List of figures

Figure 1. End-to-end workflow: how data collection, Units 1 to 3 and the dashboard connect. 2
Figure 2. Negative-review share by playtime at review and by price inside the 2-hour window (Python/Matplotlib; src/analysis, notebook 02_EDA_Visualizations.ipynb). 5
Figure 3. Cross-validation and test scores for all models at review level, game level and with review text (Python/Matplotlib; python -m src.models.compare). 7
Figure 4. Game-level XGBoost confusion matrices on the 299 test games (default and tuned class scales). 8
Figure 5. SHAP importance of the game-level XGBoost model (top features). 8
Figure 6. Confusion matrix of the text detector on the 120,487 held-out reviews (counts and row-normalised shares; Python/Matplotlib; python -m src.models.review_text_detail). 10
Figure 7. Complaint themes among negative reviews by playtime at review (NMF, eight topics; Python/Matplotlib; notebook 03). 10
Figure 8. The eight Strong rules with the highest lift: share of Strong games on training and test games (Python/Matplotlib; reports/results/association_rules_strong.csv). 12
Figure 9. Market explorer, default state (1,490 games). 14
Figure 10. Market explorer after selecting the tag Pixel Graphics (550 games; KPIs and charts update). 14
Figure 11. Tag rules page filtered to stable rules with lift at least 1.5 (37 of 95 rules). 15
Figure 12. Scorer explanation for a game predicted Struggling: reasons for and against, matched rules, and single changes that would help. 15
Figure 13. Patch impact by launch timing (change in positive-review share, points; Python/Matplotlib; notebooks/review2/patch_impact.ipynb). 24
Figure 14. Share of Struggling games by price band, release year, languages, DLC, achievements and review volume with 95% intervals (Python/Seaborn; notebooks/02_EDA_Visualizations.ipynb). 24

List of tables

Table 1. Business objectives, KPIs and success criteria. 1
Table 2. Representative sample of the final data: five games (top) and five reviews (bottom); full samples are in data/sample/. 3

Table 3. Mapping of key final fields to units and methods. 4
Table 4. Descriptive statistics of the main game- and review-level variables (cleaned data). 5
Table 5. Model comparison on the shared held-out test games (metadata only; guessing PR-AUC 0.106, macro-F1 0.22). Validation: 5-fold CV by game; test scored once. 7
Table 6. Text evaluation on 120,487 held-out English reviews (guessing PR-AUC 0.115). 9
Table 7. Association-rule configuration comparison (same 1,191 training and 299 test games for every row). 11
Table 8. Comparison of the two selected Unit 3 methods (their metrics are not interchangeable)... 12
Table 9. Comparison with existing studies (direct numerical comparison is not valid: different data, targets and protocols). 13
Table 10. Illustrative scenarios for a first-hour fix (hypothetical effect sizes and cost; sensitivity to the share of reviews written inside the window). 14
Table 11. Figure index and reproduction map (figures are embedded once, in Sections 1 to 7 and Appendix B). 15
Table 12. Objective-to-evidence traceability matrix. 16
Table 13. Individual contributions, dates and evidence (dates from the pull-request history). 17
Table 14. Generative-AI disclosure. 17
Table 15. Prioritised recommendations. 18
Table 16. Implementation roadmap with resources and success checks. 19
Table 17. Concise data dictionary of the cleaned tables. 22
Table 18. Key cleaning flags. 22
Table 19. Grid-search results: default (grid point 0) against tuned settings on the shared folds. 23
Table 20. Subgroup error check on the held-out test data (game scorer: macro-F1; text detector: PR-AUC; python -m src.models.subgroup_check). 23

Abbreviations

Abbreviation	Meaning
AUC	Area under the curve (ROC-AUC: receiver operating characteristic; PR-AUC: precision-recall)
CV	Cross-validation
EDA	Exploratory data analysis
FDR	False discovery rate
FP-Growth	Frequent-pattern growth (itemset mining)
LDA	Latent Dirichlet allocation

Abbreviation	Meaning
NMF	Non-negative matrix factorization
PCA	Principal component analysis
SHAP	Shapley additive explanations
SVD	Singular value decomposition
TF-IDF	Term frequency, inverse document frequency

1. Introduction, Business Objectives and Literature Review

1.1 Context and scope

Steam lets players refund a game within 14 days of purchase if they have played it for less than two hours [4]. Refunds themselves are not public, but reviews are, and a negative review written in that window is an early sign that the first hour did not convince the player. Our stakeholders are indie studios and small publishers who must decide what to build, what to fix before launch and when a patch is worth it. We kept the scope narrow: indie roguelike and roguelite games released between 2022 and 2025 with at least 10 reviews (1,861 games). All findings are associations, and revenue and refund figures are out of scope because Steam does not publish them.

Overall objective: help a studio decide which feature combinations to back, what to fix in the first hours of play, and when to patch. We ask five questions: will a review be negative and how does playtime affect this (Q1); which success tier will a game reach (Q2); which tag, price and Early Access combinations go with strong reception (Q3); what do players complain about (Q4); and how do ratings change after a patch (Q5). Table 1 lists the KPIs and Figure 1 shows how the units fit together.

Table 1. Business objectives, KPIs and success criteria.

Business objective	KPI / unit	Baseline / target	Unit / method
O1. Quantify the refund-window effect (Q1)	Negative share inside vs outside the window (percentage points)	Baseline: 10.4% overall. Target: report the gap with a 95% interval	U1
O2. Detect negative reviews (Q1)	PR-AUC on held-out games	Baseline: guessing, 0.115. Target: clearly above baseline, CV spread reported	U2, U3 M1
O3. Predict the game tier (Q2)	Macro-F1 and recall per tier on held-out games	Baseline: 0.22. Target: clearly above baseline	U2
O4. Find tag combinations tied to Strong reception (Q3)	Rules passing FDR control ($q < 0.10$, lift at least 1.2) and holding on test games	Baseline: none. Target: rules with a held-out check	U3 M2
O5. Find complaint themes (Q4)	Topic diversity, coherence, stability and agreement with a manual check	Baseline: LDA topic model. Target: interpretable, stable themes	U3 M1
O6. Measure patch impact (Q5)	Change in positive-review share against matched no-patch controls (points)	Baseline: 0 points. Target: interval around the difference	U1, extra
O7. Deliver an interactive application	Public dashboard with filters that update results	Target: working link, 5 pages, live scenario	App

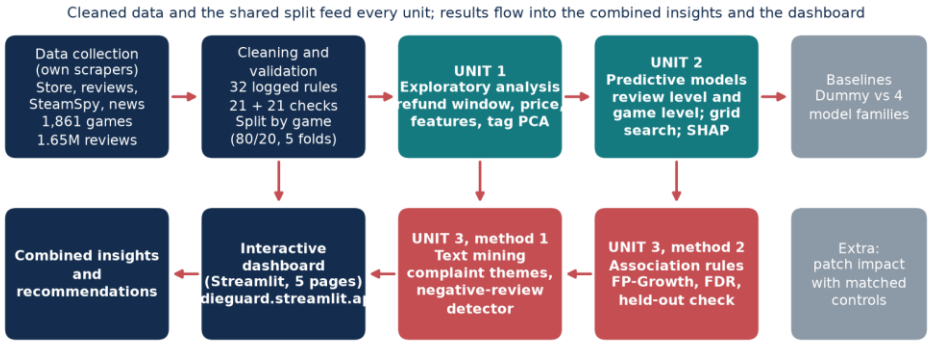


Figure 1. End-to-end workflow: how data collection, Units 1 to 3 and the dashboard connect.

1.2 Related analytics work and research gap

Lin et al. studied about 11 million reviews of 6,224 Steam games and found that negative reviews arrive early and that players criticise design more than bugs [1]. Their work is descriptive and covers the whole platform. De Luisa et al. related price, size, languages, release date and genre to a game's player numbers using Bayesian models [2]; they predict popularity rather than review sentiment and do not use review text. Jorge and Pardo built SteamBR, a corpus of 2.8 million Brazilian-Portuguese reviews, to predict which reviews are helpful [3]. Steam's own review labels and SteamSpy's owner estimates [4, 5] describe games only after the fact. None of these connects early-playtime negativity to what a studio should fix inside the refund window, or links review risk to launch features and patch timing with held-out testing. Text mining and association rules fit this gap: complaints live in free text, and launch decisions are combinations of tags, price and scope.

2. Dataset Collection, Validation and Preparation

2.1 Provenance, coverage and challenges

We collected all data ourselves on 1 and 2 October 2026 from four public Steam sources, using scrapers we wrote: store search (Indie plus five roguelike tags, 2022 to 2025), store details and tags, SteamSpy owner estimates, and the news feed (patch and sale announcements). Reviews were paged with Steam's cursor. The dataset is not taken from Kaggle or any other repository, and it contains no synthetic records. The usage terms we followed were these. The Steam Web API Terms of Use [22] limit use to 100,000 calls per day; our use is non-commercial academic analysis, and at one request every 1.25 to 1.6 seconds the scrapers stayed below that limit. The review endpoint we used is the one Steam documents publicly for listing a game's reviews [4]. The SteamSpy API asks clients to stay under one request per second [5], and our scraper waited 1.1 seconds between calls. Reviews are written by Steam users, so we keep no names or profile links and hash reviewer IDs.

The raw files are kept only on team machines (they are excluded from the public repository), frozen with row counts and a hash, and are not redistributed; the repository holds the cleaned, pseudonymised tables.

The raw data hold 1,861 games (58 variables), 1,652,999 reviews in 31 languages and 29,918 news events. After cleaning, 1,648,387 reviews (34 variables) remain. The data are observed at three levels: review, game and announcement. There are two targets. At review level the target is whether a review is negative (about 10% are). At game level it is a three-level tier, defined for the 1,490 games with at least 20 reviews: Struggling (30% or more negative), Solid, and Strong (10% or less). Table 2 shows sample rows and Appendix A gives the data dictionary.

Table 2. Representative sample of the final data: five games (top) and five reviews (bottom); full samples are in data/sample/.

Game	Release year	Price	Steam reviews	Negative share	Tier
Cross The World	2025	$19.99	23	0%	Strong
Bloody Heaven 2	2024	$4.99	138	15%	Solid
Firework War	2025	$0.00	55	4%	Strong
Godrop	2024	$8.99	313	5%	Strong
From The Top	2025	$3.99	148	16%	Solid

Review text (shortened)	Language	Playtime at review	Target
I want Nemesis to fold me in half. 11/10 would...	english	6.8 h	positive
If you're reading comments right now to decide...	english	23.9 h	positive
great roguelike. its humerous and charming, bu...	english	3.1 h	positive
bad performance for a 2d game 5 fps in the upg...	english	0.9 h	negative
Hardness for the sake of hardness. Game has se...	english	11.6 h	negative

Collection was slower than planned. Steam started refusing requests (HTTP 429) after roughly 150 calls, so we changed the scraper to wait between requests, back off when refused and save its place so that a run could resume after a failure. SteamSpy was blocked by our network provider and had to be reached through another route. For the five largest games we kept only the newest 15,000 reviews; for every other game we collected almost all the reviews Steam reports.

2.2 Data quality, privacy and ethics

Cleaning followed 32 logged rules. We dropped only unusable rows, mainly 4,612 empty reviews (0.28% of the total), corrected 318 wrong launch dates and found no duplicate IDs. Suspicious records (very short or copy-pasted text, reviews written before launch, extreme playtimes, free copies) were flagged rather than deleted, so each analysis could choose its own rule; the counts are

in Appendix A. Missing values were few in the fields we use: playtime at review is missing for 12 reviews, and Steam leaves the achievement count empty when a game has none (22% of games), which we read as zero. The Metacritic score exists for only 40 games, so we did not use it. Because only about 10% of reviews are negative, plain accuracy is misleading: always guessing positive already scores about 89%.

We collected only public data. Reviewer IDs are replaced by a salted hash, and names, avatars and profile links are never stored, in line with the data-minimisation and pseudonymisation principles of India's Digital Personal Data Protection Act, 2023 [6]; all reported outputs are aggregated or at game level. The data carry some known biases: only about 40% of reviews are in English, games with fewer than 10 reviews are excluded, reviewers decide for themselves whether to write a review, store attributes are an October 2026 snapshot, and the five largest games are capped.

2.3 Transformations and usable dataset

Cleaning is kept separate from model-specific preprocessing. The main transformations were UTC timestamps, a winsorised copy of playtime, corrected launch dates, negation handling in text (for example *not_fun*), a PCA of the tag columns (Section 4) and TF-IDF with SVD for review text. Data are split by game, 80/20, with five cross-validation folds inside the training games, so no game appears on both sides. Everything learned from data is fitted on training games only, and an automated script runs 21 leakage and consistency checks. Table 2 shows a sample of the final data and Table 3 maps the main fields to each unit.

Table 3. Mapping of key final fields to units and methods.

Variable	Type / meaning	Origin / preparation	Used in
`voted_up`	bool: recommended or not	Steam review field; target = not voted_up	U1, U2, U3 M1
`author_playtime_at_review`	number (minutes)	Playtime when written; refund window = under 120 min	U1, U2, U3 M1
`review`, `language`	text, category	As written; English subset flagged `text_mining_ok`	U3 M1
`created`	datetime (UTC)	Converted from Unix time; days since launch, patch recency	U1, U2
`price_usd`, `is_free`, `n_dlc`	number, bool	Store details; price tier	U1, U2, U3 M2
`tags`, `genres`, `categories`	text lists	User tags (top 20); 68 PCA components; 137 rule items	U2, U3 M2
`steam_total_reviews/negative`	number	Steam totals; negative share and game tier (outcome, never a feature)	U1, U2 target
`events` (patch, sale)	event table	Steam news, classified; 29,918 rows	U1, extra

3. Exploratory Data Analysis and Visualization (Unit 1)

The exploratory work started from the two questions that the later units depend on: when negative reviews appear (Q1) and how games differ at launch (Q3). Table 4 summarises the main variables, and the figures were made in Python with Matplotlib [20] and Seaborn [21]. Most of the variables are strongly skewed. The median game has 50 Steam reviews, but the largest has more than 266,000; the median review is five words long and is written after about nine hours of play. We therefore report medians and shares rather than means, and we use logarithms or bands for counts.

Table 4. Descriptive statistics of the main game- and review-level variables (cleaned data).

Variable	n	Mean	Median	Min	Max
Game price (USD)	1,861	7.91	5.99	0.00	199.99
Steam reviews per game	1,861	1,292	50	10	266,105
Negative share per game (%)	1,861	16.8	13.8	0.0	90.0
Supported languages	1,861	6.4	3	1	103
Achievements	1,861	28.1	20	0	330
Playtime at review (hours)	1,648,375	26.1	9.4	0.1	19,081.2
Review length (words)	1,648,387	28.7	5	1	2,632

About 10.4% of reviews are negative, but the share depends strongly on playtime. Reviews written in the first two hours are 21.2% negative, against roughly 8 to 10% later (Figure 2, left), and they make up 29% of all negative reviews; the same pattern appears in 94% of the games with enough reviews. Inside this window, negativity also rises with price, from about 11% for games under $5 to 23% at $5 to $10 and 32% at $20 or more (Figure 2, right). At game level, 14.5% of games are Struggling and 35% Strong.

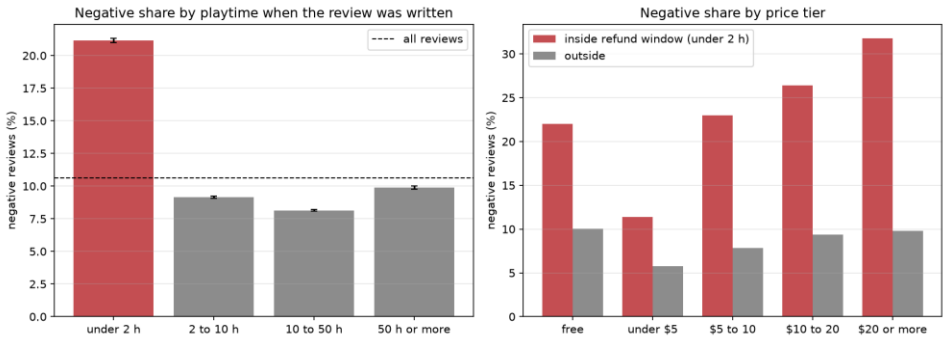


Figure 2. Negative-review share by playtime at review and by price inside the 2-hour window (Python/Matplotlib; src/analysis, notebook 02_EDA_Visualizations.ipynb).

Store attributes shift a game's risk by a few points each, but none decides it on its own (Figure 14, Appendix B). Games with more languages, DLC or achievements are less often Struggling; for example, 12.5% of games that list Steam Achievements as a store feature are Struggling, against

19.6% of those that do not. Review counts, owner estimates and refund shares are produced by a game's reception rather than known at launch, so we kept them out of the models.

These observations shaped Unit 2. Since only about 10% of reviews and 14.5% of games fall in the class we care most about, a model could reach high accuracy by ignoring them, so we evaluate with PR-AUC and macro-F1. Reviews of the same game share vocabulary and settings, so a random split would let a model recognise games instead of learning anything general; we split by game. Finally, the playtime pattern suggested that the review text itself carries the complaint, which led to the separate text model in Section 5.1.

4. Feature Engineering and Predictive Modelling (Unit 2)

At review level we predict whether a review is negative from playtime, refund-window and launch-week flags, days since launch, recent patches and sales, price and text length. At game level we predict the tier from 83 features that a studio knows at launch: price, launch year, DLC, achievements, platforms, controller support and 68 tag components. Columns that describe the outcome are never used as features. The tags needed compressing: after rare tags were dropped, 234 tag columns remained for 1,191 training games, most of them sparse and many strongly related (Pixel Graphics and Retro often appear together). A PCA fitted on the training games had its elbow at 68 components, which keep 75% of the tag variance while reducing the number of columns by more than two-thirds.

Each task has a Dummy baseline and four models: Logistic Regression, Random Forest, LightGBM and XGBoost [7, 8, 9]. All models share the same split and folds. We tuned each by grid search on the folds, keeping the default setting as one of the candidates (Appendix B, Table 19), and chose thresholds and class scales from out-of-fold predictions; the test set was scored once. For leakage, we checked that outcome columns are excluded, that no game appears on both sides of the split, and that all reviews of a game sit in a single cross-validation fold, so review-level folds and the tuning sample are grouped by game; 21 automated checks cover these points and all pass. For the metadata models, imputation and scaling are fitted inside each training fold. One exception applies to the text model in Section 5.1: its TF-IDF vocabulary and scaler were fitted once on all training reviews before cross-validation. This can make its CV scores slightly optimistic, but it does not affect the test scores, because test reviews were never used in fitting.

Table 5. Model comparison on the shared held-out test games (metadata only; guessing PR-AUC 0.106, macro-F1 0.22). Validation: 5-fold CV by game; test scored once.

Model	Review CV PR-AUC	Test PR-AUC	Test ROC-AUC	Game CV macro-F1	Test macro-F1 [95% CI]	Recall Struggling
Dummy (prior)	0.107 ± 0.024	0.106	0.500	0.225 ± 0.003	0.220 [0.20, 0.24]	0.00
Logistic Regression	0.208 ± 0.024	0.194	0.663	0.422 ± 0.033	0.423 [0.36, 0.48]	0.39
Random Forest	0.286 ± 0.038	0.280	0.726	0.434 ± 0.037	0.371 [0.31, 0.43]	0.10
LightGBM	0.287 ± 0.037	0.254	0.724	0.445 ± 0.027	0.372 [0.31, 0.43]	0.22
XGBoost	0.292 ± 0.037	0.275	0.729	0.444 ± 0.027	0.437 [0.38, 0.50]	0.41

At review level (Table 5, Figure 3) the three tree models are roughly tied at a test PR-AUC of 0.25 to 0.28, about 2.5 times guessing, while Logistic Regression is weaker (0.19). At game level every model beats guessing, but the intervals overlap because the test set has only 299 games. Tuning helped XGBoost (0.437) and Logistic Regression (0.423) on test, while Random Forest and LightGBM fell back to about 0.37 even though their cross-validation scores improved. With only 1,191 training games, the grid search probably fitted noise in the validation folds for these two models, so their higher CV scores should not be read as better generalisation; we judge all models on the test results. Struggling games are the hardest class to find (Figure 4). We selected XGBoost at game level for its test score and Struggling recall, although it is not clearly better than Logistic Regression.

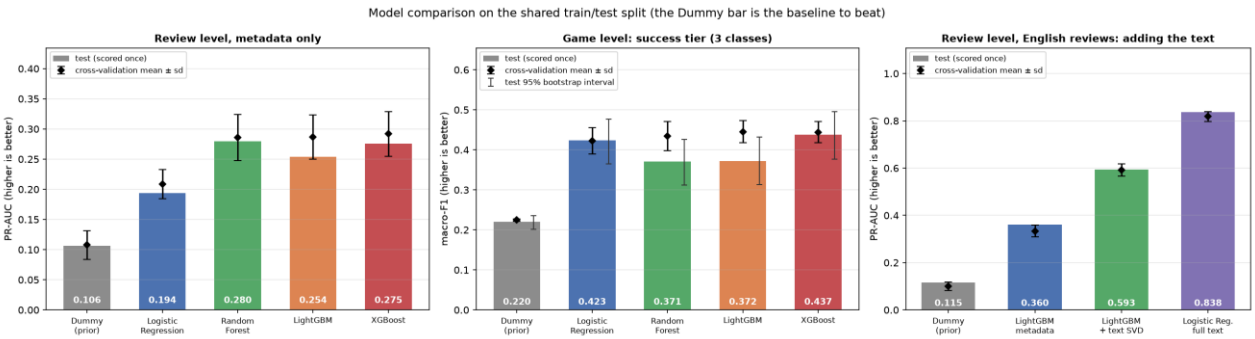


Figure 3. Cross-validation and test scores for all models at review level, game level and with review text (Python/Matplotlib; python -m src.models.compare).

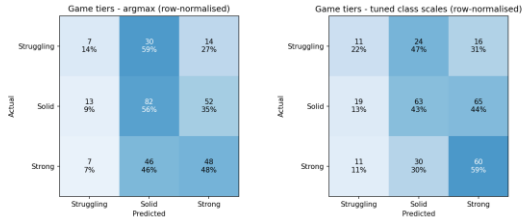


Figure 4. Game-level XGBoost confusion matrices on the 299 test games (default and tuned class scales).

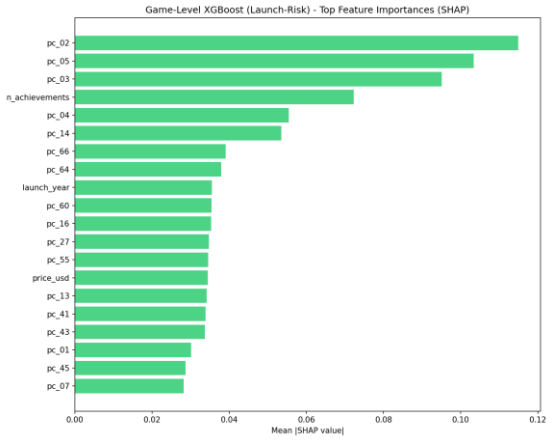


Figure 5. SHAP importance of the game-level XGBoost model (top features).

We explain the tree models with SHAP values [15]. At review level, the model leans on review length, Chinese-language reviews and playtime; at game level, on a few tag components and the number of achievements (Figure 5). These are associations, not causes, and the review-level models use reviewer history collected at scrape time, so they explain reviews rather than predict a launch. The results are not driven by a few large games: the within-game ROC-AUC (0.75) is slightly above the pooled value.

To check for uneven errors, we sliced the test predictions by subgroup (Appendix B, Table 20). For the deployed game-level scorer (macro-F1 0.43 overall), macro-F1 ranges from 0.33 for Early Access launches and 0.36 for 2022 to 2023 games to 0.47 for 2024 to 2025 games, and it finds few Struggling games among those with 100 or more reviews. These groups hold only 37 to 239 games, so the differences are uncertain, but the scorer should be used with more caution for Early Access plans. The text detector is consistent across game price bands (PR-AUC 0.83 to 0.86) and works best inside the refund window (0.91). It is weakest on very short reviews (under 10 words: PR-AUC 0.73, recall 0.64) and raises more false alarms on free games (precision 0.60), so short reviews and free games need a manual look. The text model covers English only, so we could not compare languages.

5. Advanced Analytics (Unit 3): Two Selected Methods

The EDA left two questions that the Unit 2 models could not answer. Negative reviews cluster in the first hours, but the playtime and store fields do not say what players are unhappy about; that information is in the review text, so we chose text mining. Store data also predicted the game tier only modestly, and a launch plan is a combination of tags, price and Early Access status rather than

a single feature, so we chose association rule mining to look for combinations. We did not choose time-series analysis because most games have too few reviews per week for stable series; patch impact is reported as an extra analysis in Section 6.1.

5.1 Method 1: Text mining

This method asks what players complain about, and whether the review text can flag a negative review. We used 557,202 English reviews (436,715 for training, 120,487 for testing). The text was cleaned and negations were joined to the next word, so that *not fun* becomes one token instead of counting as *fun*. We represented each review with TF-IDF [10], which gives more weight to words that are frequent in a review but rare across reviews; it suits short, informal text and works well with linear models. The vocabulary and weights were learned from training reviews only.

We built two things on this representation. The first is a detector: a Logistic Regression on TF-IDF plus the metadata features, tuned by grid search, with its threshold set on out-of-fold predictions. To see how much of the text signal survives compression, we also trained a model on 78 SVD components of the text (the elbow of the variance curve). The second is a topic model, NMF [11] with eight topics on 40,000 negative training reviews, compared with LDA [12]. We fixed eight topics so that each could be read and labelled by hand; we did not search formally over other numbers of topics.

Table 6. Text evaluation on 120,487 held-out English reviews (guessing PR-AUC 0.115).

Approach	CV PR-AUC	Test PR-AUC	Test ROC-AUC	Precision / recall	F1
Dummy (prior)	0.101 ± 0.018	0.115	0.500	0.12 / 1.00	0.21
LightGBM, metadata only	0.334 ± 0.025	0.360	0.758	0.32 / 0.49	0.39
LightGBM, metadata + text components	0.592 ± 0.026	0.593	0.895	0.52 / 0.62	0.56
Logistic Regression, full text + metadata	0.819 ± 0.021	0.838	0.971	0.75 / 0.79	0.77

Compressed text features lift PR-AUC from 0.36 to 0.59, and the full TF-IDF model reaches 0.84 (ROC-AUC 0.97). On the test reviews it catches 10,980 negative reviews and misses 2,893, with 3,607 false alarms (Figure 6); accuracy is 94.6% against 88.5% for always guessing positive. For the topics, NMF scored lower than LDA on UMass coherence (-2.38 against -1.73). UMass rewards words that often appear together, and on short reviews LDA reached a higher score by repeating the same general gaming words across several topics. NMF topics overlapped less (90% of their top words were unique, against 86% for LDA), and when we refitted NMF on 80% resamples, the top words of most topics stayed similar (mean Jaccard 0.75). We therefore used NMF because its topics were easier to tell apart. Early negative reviews lean towards crashes, multiplayer bugs and controls,

later ones towards boss design and balance (Figure 7); a word such as *boring* appears in 9.1% of negative reviews against 0.8% of positive ones.

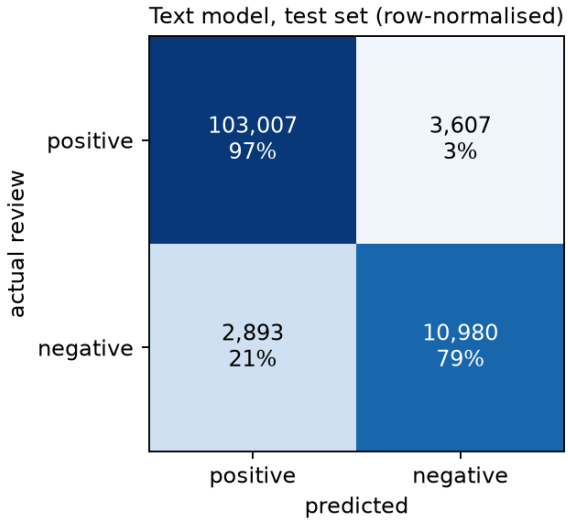


Figure 6. Confusion matrix of the text detector on the 120,487 held-out reviews (counts and row-normalised shares; Python/Matplotlib; python -m src.models.review_text_detail).

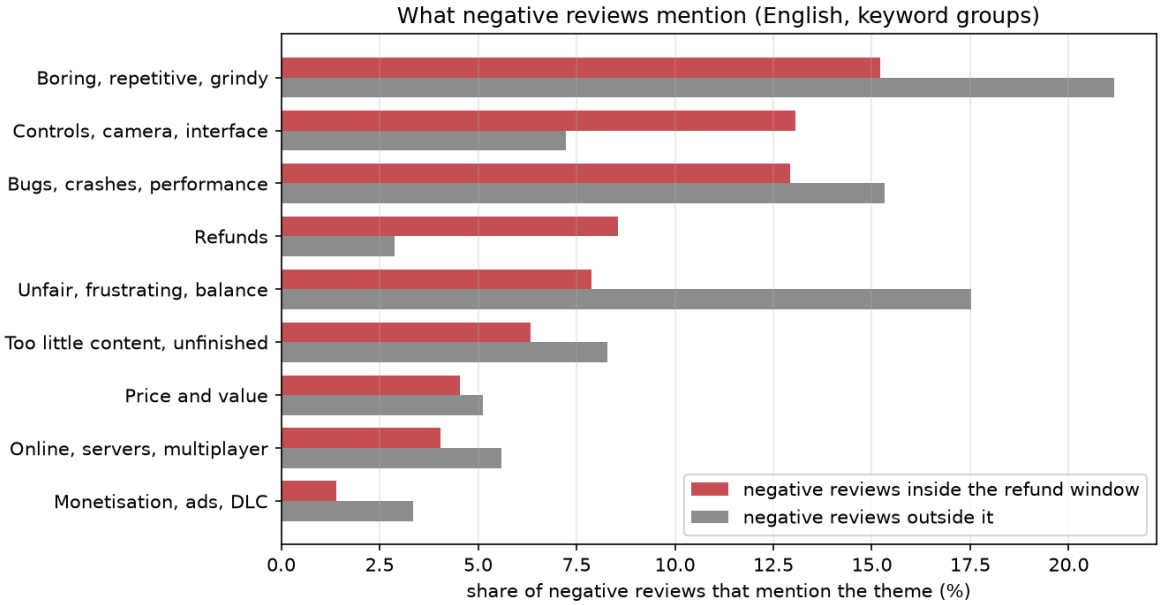


Figure 7. Complaint themes among negative reviews by playtime at review (NMF, eight topics; Python/Matplotlib; notebook 03).

In a blind check of 200 reviews, labelled by an AI assistant without seeing the model's topic, the labels matched the model on 35.5% (kappa 0.30). The Early Access, repetition and crash topics were recovered well; the boss and balance topics were not, so we treat those as exploratory. An earlier figure of 96% was withdrawn because those labels were not independent; a human re-label

would make the check stronger. The detector identifies complaints in existing reviews; it does not forecast them.

5.2 Method 2: Association rule mining

This method asks which tag, price and Early Access combinations go with the Strong tier. Each of the 1,191 training games became a transaction of up to 137 possible items: its user tags, a price band and Early Access. We left out opinion tags such as *Great Soundtrack*, because players add them after playing and they would make the rules circular. We mined frequent itemsets with FP-Growth [13] using mlxtend [16], keeping itemsets of up to three items that appear in at least 25 games (about 2% of the training games). Below that support, a rule rests on so few games that it rarely holds on new ones (Table 7).

Three measures describe a rule. Take the strongest rule, *Pixel Graphics + Retro + Shooter leads to Strong*. Its support is the 25 training games that have all three tags. Its confidence is the share of those games that are Strong, 20 of 25 or 80%. Its lift compares that with the base rate: 80% against 35% of all training games, a lift of 2.26. From 4,715 candidate rules we kept those with a lift of at least 1.2 that also passed a false-discovery control (Benjamini-Hochberg, q below 0.10) [14], which removes rules likely to appear by chance among so many tests. For the held-out check, we took the test games that contain all of a rule's items. A rule holds if their Strong share is still above the test base rate (lift above 1) and its training confidence lies inside the 95% interval of the test confidence. Rules that match fewer than 10 test games could not be tested.

Table 7. Association-rule configuration comparison (same 1,191 training and 299 test games for every row).

Configuration	Strong rules (FDR)	Mean confidence / lift	Hold on test games
FP-Growth, support 25, up to 3 items (chosen)	95	0.59 / 1.68	68% of 66
Apriori, same thresholds	95	0.59 / 1.68	68% of 66
FP-Growth, support 15 (looser)	107	0.65 / 1.85	66% of 56
FP-Growth, support 40 (stricter)	69	0.57 / 1.60	69% of 59
FP-Growth, pairs only	41	0.56 / 1.59	83% of 30
FP-Growth, keeping after-play tags	131	0.60 / 1.69	69% of 88

Of 4,715 candidate rules, 95 pass for the Strong tier and none for Struggling: under these thresholds, no tag combination is clearly associated with a struggling launch. About two-thirds of the rules that could be tested (45 of 66) still hold on unseen games (Figure 8). FP-Growth and Apriori return the same rules, and a lower support finds more rules that hold less often (Table 7).

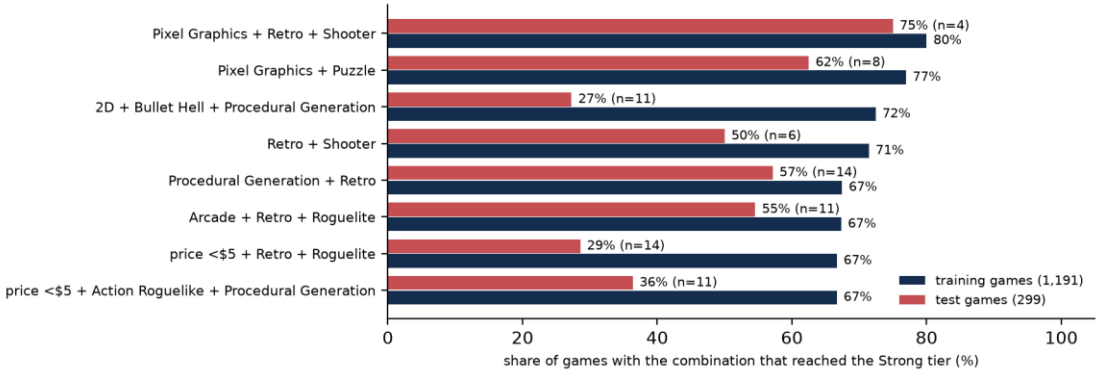


Figure 8. The eight Strong rules with the highest lift: share of Strong games on training and test games (Python/Matplotlib; reports/results/association_rules_strong.csv).

Each rule rests on only 25 to 60 training games and a handful of test games, and rules involving a price under $5 are often unstable. They are hypotheses for a studio to test, not recipes, and they say nothing about why the combinations do well.

6. Comparative Analysis and Business Impact

6.1 Comparison of the two Unit 3 methods

Table 8. Comparison of the two selected Unit 3 methods (their metrics are not interchangeable).

Criterion	Method 1: text mining	Method 2: association rules
Question	What do players complain about; can text flag a negative review?	Which launch combinations go with Strong reception?
Input and output	557,202 English reviews; detector scores and eight themes	1,191 training games as itemsets; 95 Strong rules
Validation	Held-out reviews from unseen games; PR-AUC, ROC-AUC; topic diversity and stability; blind check	FDR control; each rule checked once on 299 test games; stability
Strengths	Strong, well-validated detector (ROC-AUC 0.97); themes explain complaints	Transparent rules a studio can read; launch-time information only
Limits	Detects, does not forecast; weak topic agreement (35.5%); English only	Small supports; no Struggling rule; associations only

The two methods answer different questions, so their metrics cannot be compared directly: PR-AUC measures how well the detector separates negative from positive reviews, while lift and the held-out share measure how far a tag combination raises the chance of a Strong game. Text mining describes the complaints players make and when they make them; the rules describe launch choices that are associated with good reception. The findings line up across units: complaints cluster in the first two hours and at higher prices (Unit 1; crashes and controls in the text), while store data and tags only partly separate good launches from weak ones (Unit 2 and the rules). As an extra analysis, positive reviews fall by about 2 points after a patch, but matched games without a patch fall by almost the same amount (difference +0.27 points, interval -0.82 to +1.36). We therefore find no

evidence of a patch effect of its own; the drop is consistent with ratings settling after launch (Appendix B, Figure 13).

6.2 Comparison with existing studies

Table 9. Comparison with existing studies (direct numerical comparison is not valid: different data, targets and protocols).

Criterion	Lin et al. [1]	De Luisa et al. [2]	SteamBR [3]	Current project
Data and setting	About 11M reviews of 6,224 Steam games (2016 snapshot), whole platform	Steam games; popularity as player numbers	2.8M Brazilian-Portuguese Steam reviews	1,861 indie roguelikes, 1.65M reviews, Oct 2026
Methods and validation	Descriptive analysis; no model scored on unseen games	Bayesian hierarchical models; coefficients	Text classifiers for review helpfulness	Models scored once on held-out games; baselines; FDR-controlled rules; topics
Comparable findings	Negative reviews come early (we: 21.2% inside 2 h); design criticised more than bugs (we: crashes early, balance later)	Price, languages and release date matter for popularity; our tier macro-F1 is modest (0.42 to 0.44)	Review text can be modelled; ours detects negativity (PR-AUC 0.84), a different target	Adds refund window, patch timing, text and tags in one niche
Why not numeric	No predictive metric	Different target (players, not tiers)	Helpfulness, other language	Gaps: causal claims, revenue, other genres

None of the three studies can be compared with ours on a single number, because each uses different data, a different target and a different evaluation. The closest is Lin et al., whose finding that negative reviews arrive early matches our refund-window result in direction. SteamBR shows that Steam review text can be modelled reliably with text classifiers, which supports our text-based approach, although it predicts helpfulness in Portuguese rather than negativity in English. Our contribution is a whole niche collected from scratch, the refund-window view, held-out testing that also reports weak results, and an application a studio can use.

6.3 KPI-based business impact and scenarios

We turned the refund-window finding into an illustrative scenario for one game. Three numbers are measured: about 14% of reviews are written in the first two hours, 21.2% of those are negative, and the figure is 11.4% for the cheapest games. The rest is assumed: a game with 1,000 reviews; a first-hour fix (blind testing, crash and control fixes) taking about 40 person-hours at an illustrative $30 per hour (our assumption for easy arithmetic, not an industry rate), or about $1,200; and a drop in early negativity of 0, 5 or 9.8 points (the last matches the cheapest tier). Sales and refunds are not public, so we cannot put a money value on a review. Instead, the table shows what each avoided negative review would cost, which a studio can compare with what a review is worth to it.

Table 10. Illustrative scenarios for a first-hour fix (hypothetical effect sizes and cost; sensitivity to the share of reviews written inside the window).

Scenario	Change in early negativity	Avoided negatives per 1,000 reviews (window share 10% / 14% measured / 20%)	Overall negative share (14%)	Cost per avoided negative ($1,200)
Worst	-0.0 points	0.0 / 0.0 / 0.0	10.4% to 10.4%	no benefit
Base	-5.0 points	5.0 / 7.0 / 10.0	10.4% to 9.7%	$171
Best	-9.8 points	9.8 / 13.7 / 19.6	10.4% to 9.0%	$88

The gain is modest: about 7 fewer negative reviews per 1,000 in the base case, at roughly $171 each. The net impact is positive only if a negative review avoided early is worth more than that to the studio in kept sales and rating, which our data cannot show. The fix pays off more easily for games that expect many reviews, because the cost is fixed while the benefit grows with the number of reviews. The estimate also depends on how much a fix achieves and how many reviews come early. For patches, we found no general effect, so patching only to lift sentiment is not supported by our data; patching specific crashes and controls still makes sense. Section 9 ranks the recommendations.

7. Interactive Application, Figure Index and Reproducibility

7.1 Mandatory dashboard

The dashboard is public at https://indieguard.streamlit.app. The free host sleeps when idle, so it should be opened a few minutes before a demo. It can also be run locally with `streamlit run src/dashboard/app.py`. It is built with Streamlit [17], pandas [18] and Altair [19], and reads small tables exported from the analysis and one saved XGBoost model. Unit 1 summaries and the Unit 3 rule, topic and patch results are precomputed; filters, charts and the launch-risk scorer are computed live. It has five pages: market explorer, tag rules and complaints, patch impact, launch-risk scorer and an about page.

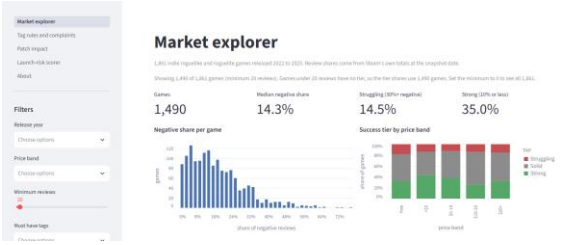


Figure 9. Market explorer, default state (1,490 games).

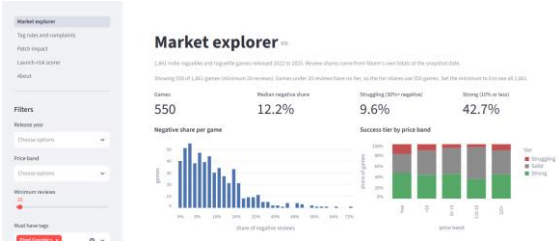


Figure 10. Market explorer after selecting the tag Pixel Graphics (550 games; KPIs and charts update).



Figure 11. Tag rules page filtered to stable rules with lift at least 1.5 (37 of 95 rules).



Figure 12. Scorer explanation for a game predicted Struggling: reasons for and against, matched rules, and single changes that would help.

At Review 2 the panel asked that the scorer explain its output, so we added an explanation step. After scoring, the dashboard shows why a game looks strong or weak. Each launch choice (achievements, controller support, a tag and so on) is replaced by the opposite choice, for example no achievements instead of 30, and the game is scored again; the page then lists the choices that lower and those that raise the risk of a Struggling launch. It also lists the tag rules the game matches and suggests single changes that would most improve the outlook (Figure 12). For testing, we confirmed that the saved feature pipeline reproduces the stored features for all 1,861 games, and opened every page both in the deployed app and in an automated browser run. The main limits are that the scores are class-scaled, not probabilities (about 0.43 macro-F1), the explanations are associations, and the data are an October 2026 snapshot.

7.2 Figure index and reproduction map

Table 11. Figure index and reproduction map (figures are embedded once, in Sections 1 to 7 and Appendix B).

Figure	Title / tool	Section / purpose	Source notebook / file
Fig. 1	End-to-end workflow, Matplotlib	1.1: how units connect	`build_capstone_report.py`
Fig. 2	Refund window and price, Matplotlib	3: core Unit 1 finding	`figures/findings/refund_window.png`; notebook 02
Fig. 3	Model comparison, Matplotlib	4: models vs baselines	`python -m src.models.compare`
Fig. 4, 5	Game confusion; SHAP bars	4: errors, interpretation	`figures/models/game_xgb_*.png`
Fig. 6, 7	Text confusion; complaint themes	5.1: detector and themes	`review_text_detail.py`; notebook 03
Fig. 8	Top rules train vs test, Matplotlib	5.2: held-out check	`association_rules_strong.csv`
Fig. 9 to 12	Dashboard screenshots	7.1: interactive states	`figures/dashboard/`

Figure	Title / tool	Section / purpose	Source notebook / file
Fig. 13, 14	Patch impact; risk by game features	App. B: 6.1 extra; 3 store signals	`patch_impact.ipynb`; `figures/eda/10_*.png`

7.3 Code and environment

The code is in a public repository, https://github.com/Shan713/IndieGuard. Its README describes the folder layout (`src`, `notebooks`, `data`, `figures`, `reports`, `docs`, `models`, `scripts`). The project runs on Python 3.12, with the package versions pinned in `requirements.txt` and installed with `pip install -r requirements.txt`. The commands that rebuild the features, models, tuning searches, report and dashboard are listed in Appendix C. No secrets are stored; the raw scrape and the hashing salt are left out, and the collection scripts can regenerate the data from Steam's public endpoints within Steam's terms.

8. End-to-End Validation, Traceability and Contributions

8.1 Completion and validity

Everything in the declared scope is implemented and can be shown: data collection and cleaning, EDA, five model families at review and game level with tuning, two Unit 3 methods, the deployed dashboard, this report and the presentation. The main threat to internal validity is the small number of games available for tuning: with 1,191 training games, the grid search appears to have overfitted the validation folds for two models, and the topic check relied on a single AI annotator. External validity is limited because the data cover one niche at one point in time, only about 40% of reviews are in English, and there is no revenue data, so the results may not carry over to other genres. On construct validity, the review models use reviewer history collected at scrape time, and the game tier is a rule on the negative share rather than a business outcome. Results are reproducible from the scripts, although game-level scores move by about 0.01 between runs. The subgroup checks covered the game scorer and the English text detector only, and discounts were not analysed.

Table 12. Objective-to-evidence traceability matrix.

Objective / KPI	Analysis / method	Result / metric	Figure / artifact	Recommendation
O1 Refund window	Section 3, EDA	21.2% vs 8.8% negative; 29% of negatives	Fig. 2	R1, R2
O2 Detect negative reviews	Sections 4, 5.1	PR-AUC 0.838; ROC-AUC 0.971 (guessing 0.115)	Fig. 3, 6; Table 6	R1
O3 Predict game tier	Section 4	Macro-F1 0.42 to 0.44 vs 0.22	Fig. 3, 4; Table 5	R4
O4 Tag combinations	Section 5.2	95 Strong rules; 68% of 66 testable hold; none for Struggling	Fig. 8; Table 7	R3

Objective / KPI	Analysis / method	Result / metric	Figure / artifact	Recommendation
O5 Complaint themes	Section 5.1	NMF diversity 90.0%; blind check 35.5%	Fig. 7	R1, R5
O6 Patch impact	Section 6.1	+0.27 points [-0.82, +1.36] vs controls	Fig. 13	R5
O7 Application	Section 7	Public dashboard, 5 pages, filters and scorer	Fig. 9 to 12	R1 to R5 (exploration)

8.2 Project tracking and individual work

Work was tracked on a GitHub Projects board (https://github.com/users/Shan713/projects/1, private, access on request) with the stages Backlog, To Do, In Progress, Review/Testing and Completed; each card has an owner, a due date and evidence. Commits and pull requests are public in the repository. Individual marks may vary with participation and the explanation of assigned work.

Table 13. Individual contributions, dates and evidence (dates from the pull-request history).

Team member / register no.	Assigned work	Period	Evidence link / artifact
Thotakura Srihitha / CB.SC.U4CSE23450	Exploratory analysis (29 figures, association tests, tag combinations); dashboard wireframe and downloads; grid-search tuning of the game-level models and review Logistic Regression	2 to 17 Oct 2026	PRs #1, #101, #102; notebook 02; cards #20, #21, #37, #40, #41, #86 to #89, #91, #93
Gayas Sheik / CB.SC.U4CSE23118	Feature engineering and tag PCA; patch impact with matched controls; launch-risk scorer; reproducibility	2 to 14 Oct 2026	PRs #2, #49, #50, #69, #71, #80; cards #4, #18, #19, #38, #39, #42
Shantharam P / CB.SC.U4CSE23146	Data collection and cleaning; split and targets; grid-search tuning of the game-level models and review Logistic Regression; dashboard and deployment	1 to 17 Oct 2026	`src/collect`, `src/clean`; PRs #57, #95, #96, #102; cards #5 to #17, #23 to #27, #43, #83 to #94
Pradeep Sasidharan / CB.SC.U4CSE23232	Model development: review- and game-level models (Logistic Regression, Random Forest, XGBoost) with SHAP; model comparison against baselines; association rule mining	3 to 11 Oct 2026	`src/models`; PRs #55, #70; cards #34, #35, #36
Valikala Tejaswini / CB.SC.U4CSE23752	Text mining and topic modelling; combined business insights; blind topic check	10 to 16 Oct 2026	PRs #66, #81, #99; cards #31, #32, #33, #44

8.3 AI use and originality declarations

We list every generative-AI tool we used. The tools assisted the team; they did not replace it. The research question, scope, data collection runs, analytical choices and conclusions were decided by team members, and each member wrote and checked their own modules. By signing the declaration page, each member confirms that this list is complete for their own work.

Table 14. Generative-AI disclosure.

Tool	Tasks supported	Content / code affected	Human verification and corrections

Tool	Tasks supported	Content / code affected	Human verification and corrections
ChatGPT (OpenAI)	Ideation: brainstorming project topics, research questions and feature ideas in the planning stage.	Early ideas and the shortlist of topics and questions. No data, code or reported results.	The team discussed the ideas, checked feasibility against the available public data, and wrote and agreed the problem statement and KPIs. Nothing from the tool is reported as a result.
Antigravity (AI coding assistant)	Minor coding assistance: small code completions and fixes in project scripts.	Small, scattered completions and fixes; no complete module or analysis.	Each change was read, run and tested by the team member who made it; outputs were checked against expected values and the automated checks (Section 8.1).
Claude (Anthropic)	Coding and debugging help under the team's direction (data collection, modelling, dashboard and report scripts); drafting documentation, this report and the presentation deck; labelling the blind topic-check sample.	Parts of the project scripts, documentation, slides and this report.	The team ran, checked and corrected all outputs: automated checks, results regenerated from scripts, pull-request review, and slides and figures checked against the data. Errors found and fixed included a text-processing bug and an overstated topic-agreement figure, which was withdrawn.

No AI tool collected data on its own, created or altered records, or decided the conclusions; every figure in this report is produced by code from the project data. Apart from the cited sources, the ideas, analysis and conclusions are the team's own. Where AI tools produced draft text or code, as listed above, the team reviewed, edited and verified it before it was used, and borrowed ideas, figures and code are cited.

9. Conclusion, Recommendations and Roadmap

The traceability matrix (Table 12) shows how each objective was met. We quantified the refund-window effect, built a text detector and complaint themes, predicted game tiers modestly, found and tested tag-combination rules, measured patch impact against controls, and built a public dashboard. Three results were weaker than we hoped: game-tier prediction from store data is modest, no rule was found for struggling games, and patches showed no effect. These still matter to a studio, because they show where store data and patching alone are not enough.

Table 15. Prioritised recommendations.

Recommendation (hypothesis)	KPI effect	Stakeholder	Priority: impact and feasibility
R1. Treat the first hour as a standalone product: blind-test the first 30 minutes, check controls and camera, enable crash reporting on launch day	Early negative share (21.2%)	Studio / QA lead	High impact, high feasibility
R2. Match the price to what the first hour delivers: early negativity doubles from under $5 to $5 to $10, so a higher price needs a first hour that testers rate well	Early negativity by price band (11.4% to 31.8%)	Studio / publisher	Medium impact, high feasibility (test it)
R3. Test a focused 2D, pixel, retro or procedural scope before committing to multiplayer or 3D	Share of Strong games (about 2/3 vs 35%)	Design lead	Medium impact, medium feasibility

Recommendation (hypothesis)	KPI effect	Stakeholder	Priority: impact and feasibility
R4. Test whether a complete store page (achievements, controller support) helps: games with these features struggle less often, but this is an association, not proof that adding them reduces risk	Share of Struggling games (12.5% vs 19.6%, observational)	Marketing / producer	Low to medium impact, high feasibility
R5. Patch for specific problems (crashes, controls, balance), not to lift sentiment	Change in positive share (+0.27 points vs controls)	Engineering	Medium impact, high feasibility

The roadmap below turns the recommendations into steps, each with an owner, the resources it needs and a check that shows whether it worked.

Table 16. Implementation roadmap with resources and success checks.

Stage	Steps	Owner	Resources	Success check
Now (first month)	Apply R1 to the next build; run the scorer on the planned scope; small price test for R2	Studio QA lead, producer	About 40 person-hours; 10 to 20 blind testers; the dashboard	Early negative share below 21% in the first launch month
Next (1 to 6 months)	Refresh the Steam data; hand-label 200 reviews; re-check R3 and R4 on new releases	Analytics team	Scrapers (about 2 days of runtime); 2 people for labelling	Topic agreement above 35.5%; at least 2/3 of rules still hold
Later (6 to 12 months)	Calibrate the scorer; add discount analysis and other genres	Analytics team	New data collection; model retraining	Target: scorer macro-F1 of at least 0.43 on new games (not yet shown); calibrated probabilities

Several extensions were outside the scope of this project: the impact of discounts, causal designs such as experiments, revenue analysis, a language comparison for the review models and a human-labelled topic check.

References

[1] Dayi Lin, Cor-Paul Bezemer, Ying Zou, and Ahmed E. Hassan. 2019. An empirical study of game reviews on the Steam platform. *Empirical Software Engineering* 24, 1 (2019), 170-207. https://doi.org/10.1007/s10664-018-9627-4

[2] Andraž De Luisa, Jan Hartman, David Nabergoj, Samo Pahor, Marko Rus, Bozhidar Stevanoski, Jure Demšar, and Erik Štrumbelj. 2021. Predicting the popularity of games on Steam. *Elektrotehniški vestnik* 88, 4 (2021), 151-162. arXiv:2110.02896.

[3] Gabriel A. Z. Jorge and Thiago A. S. Pardo. 2023. SteamBR: a dataset for game reviews and evaluation of a state-of-the-art method for helpfulness prediction. In *Proceedings of the Brazilian Workshop on Social Network Analysis and Mining (BraSNAM)*.

[4] Valve Corporation. Steam Refunds. Steam Support. https://store.steampowered.com/steam_refunds/ (accessed October 2026); and User Reviews, Steamworks documentation. https://partner.steamgames.com/doc/store/getreviews (accessed October 2026).

[5] SteamSpy. SteamSpy API: owner estimates and game statistics, with request-rate limits. https://steamspy.com/api.php (accessed October 2026).

[6] Government of India. 2023. The Digital Personal Data Protection Act, 2023 (No. 22 of 2023).

[7] Fabian Pedregosa et al. 2011. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825-2830.

[8] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*.

[9] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: a scalable tree boosting system. In *Proceedings of KDD '16*. ACM, 785-794. https://doi.org/10.1145/2939672.2939785

[10] Gerard Salton and Christopher Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management* 24, 5 (1988), 513-523.

[11] Daniel D. Lee and H. Sebastian Seung. 1999. Learning the parts of objects by non-negative matrix factorization. *Nature* 401 (1999), 788-791.

[12] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet allocation. *Journal of Machine Learning Research* 3 (2003), 993-1022.

[13] Jiawei Han, Jian Pei, and Yiwen Yin. 2000. Mining frequent patterns without candidate generation. In *Proceedings of SIGMOD '00*. ACM, 1-12. https://doi.org/10.1145/335191.335372

[14] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289-300.

[15] Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*.

[16] Sebastian Raschka. 2018. MLxtend: providing machine learning and data science utilities and extensions to Python's scientific computing stack. *Journal of Open Source Software* 3, 24 (2018), 638.

[17] Streamlit Inc. Streamlit: the fastest way to build data apps. https://streamlit.io (version 1.66, accessed October 2026).

[18] Wes McKinney. 2010. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, 56-61. https://doi.org/10.25080/Majora-92bf1922-00a

[19] Jacob VanderPlas, Brian E. Granger, Jeffrey Heer, Dominik Moritz, Kanit Wongsuphasawat, Arvind Satyanarayan, Eitan Lees, Ilia Timofeev, Ben Welsh, and Scott Sievert. 2018. Altair: interactive statistical visualizations for Python. *Journal of Open Source Software* 3, 32 (2018), 1057. https://doi.org/10.21105/joss.01057

[20] John D. Hunter. 2007. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering* 9, 3 (2007), 90-95. https://doi.org/10.1109/MCSE.2007.55

[21] Michael L. Waskom. 2021. seaborn: statistical data visualization. *Journal of Open Source Software* 6, 60 (2021), 3021. https://doi.org/10.21105/joss.03021

[22] Valve Corporation. Steam Web API Terms of Use. https://steamcommunity.com/dev/apiterms (accessed October 2026).

In-text citation numbers follow ACM numeric style.

Appendix A. Extended data dictionary and validation records

Table 17 lists the main fields of the three cleaned tables. The full dictionary (all 34 review columns, 58 game columns and the event table) is in `docs/data_documentation.md`, section 5, and the feature catalogue is `docs/feature_engineering/feature_catalog.csv`.

Table 17. Concise data dictionary of the cleaned tables.

Table	Field	Type	Meaning
Review	`recommendationid`	int	Steam's review ID (primary key)
Review	`appid`	int	Game ID; joins to the game table
Review	`voted_up`	bool	Recommended (True) or not; the review target is not voted_up
Review	`review`, `language`	text, category	Review text as written; 31 languages (39.5% English)
Review	`author_playtime_at_review`	float (minutes)	Playtime when the review was written (12 missing)
Review	`created`	datetime (UTC)	When the review was written
Review	`steam_purchase`, `received_for_free`	bool	Bought on Steam; marked as received for free (2.1%)
Review	`written_during_early_access`	bool	Written while the game was in Early Access (22.7%)
Review	`author_steamid_hash`	text	Salted SHA-256 of the reviewer ID (pseudonymised)
Game	`appid`, `name`	int, text	Steam app ID and name
Game	`price_usd`, `is_free`	float, bool	Regular price in USD (0 if free); free to play (308 games)
Game	`launch_date`, `release_date`	datetime	Original launch including Early Access; store (1.0) release
Game	`tags`, `genres`, `categories`	text lists	Top 20 user tags; Steam genres; store features
Game	`n_dlc`, `n_achievements`, `n_supported_languages`	int	Counts at collection time (missing achievements = none)
Game	`platform_mac`, `platform_linux`, `controller_support`	bool, text	Platforms; full or partial controller support
Game	`steam_total_reviews`, `steam_total_negative`	int	Steam's all-language totals; used for the tier, never as features
Game	`spy_owners_low`, `spy_owners_high`	int	SteamSpy owner range (outcome; not a feature)
Event	`appid`, `date`, `event_type`	int, datetime, text	Steam news item: patch or sale announcement

Validation records are in `docs/cleaning_log.md`, `docs/validation_summary.md` and `data/processed/cleaning_log.json`. Key flags added by cleaning:

Table 18. Key cleaning flags.

Field	Type	Meaning and count
`playtime_outlier`	bool	Playtime at review above the 99.5% quantile (363 h); 8,242 reviews
`is_prerelease`	bool	Written before the game's launch; 30,018 reviews
`low_content`	bool	Under 3 letters or 2 words; 208,724 reviews
`is_ascii_art`, `is_copypasta`	bool	ASCII art (4,951); same 50+ character text 10+ times (2,285)
`text_mining_ok`	bool	English and none of the three flags: the text-mining subset
`received_for_free`	bool	Reviewer received the game for free; 34,394 reviews
`launch_date` vs `release_date`	datetime	Original launch vs 1.0 date; differ for 318 games
`reviews_capped`	bool	Newest ~15,000 reviews only (5 games)

Appendix B. Tuning, additional figures and supplementary analysis

Table 19. Grid-search results: default (grid point 0) against tuned settings on the shared folds.

Search	Metric	Grid points	Default CV	Tuned CV
Review level, Logistic Regression	PR-AUC	11	0.208	0.208
Review level, Random Forest	PR-AUC	13	0.278	0.281
Review level, LightGBM	PR-AUC	25	0.281	0.287
Review level, XGBoost	PR-AUC	37	0.285	0.292
Game level, Logistic Regression	macro-F1	12	0.418	0.422
Game level, Random Forest	macro-F1	60	0.354	0.434
Game level, LightGBM	macro-F1	55	0.417	0.445
Game level, XGBoost	macro-F1	97	0.409	0.444
Review text, Logistic Regression	PR-AUC	30	0.792	0.794
Review text, LightGBM metadata	PR-AUC	25	0.332	0.334
Review text, LightGBM metadata + text	PR-AUC	25	0.574	0.592

Table 20. Subgroup error check on the held-out test data (game scorer: macro-F1; text detector: PR-AUC; python -m src.models.subgroup_check).

Model	Subgroup	Test n	Macro-F1 / PR-AUC	Struggling recall (n) / precision, recall
Game scorer	all: all test games	299	0.43	0.29 (51)
Game scorer	price: under $5	129	0.42	0.36 (25)
Game scorer	price: $5 to $14.99	133	0.41	0.19 (16)
Game scorer	price: $15 or more	37	0.45	0.30 (10)
Game scorer	reviews: 20 to 99	171	0.40	0.38 (37)
Game scorer	reviews: 100 to 499	65	0.30	0.09 (11)
Game scorer	reviews: 500 or more	63	0.45	0.00 (3)
Game scorer	year: 2022-2023	129	0.36	0.29 (24)
Game scorer	year: 2024-2025	170	0.47	0.30 (27)
Game scorer	early access: Full release	239	0.44	0.31 (45)
Game scorer	early access: Early Access	60	0.33	0.17 (6)
Text detector	window: inside 2 h	15,316	0.91	0.79 / 0.89
Text detector	window: after 2 h	105,171	0.80	0.73 / 0.75
Text detector	price: under $5	4,884	0.86	0.78 / 0.79
Text detector	price: $15 or more	71,864	0.83	0.77 / 0.77
Text detector	price: $5 to $14.99	40,176	0.86	0.74 / 0.84
Text detector	price: free	3,563	0.83	0.60 / 0.87
Text detector	length: 10 to 49 words	49,846	0.81	0.75 / 0.73
Text detector	length: under 10 words	33,022	0.73	0.72 / 0.64
Text detector	length: 50+ words	37,619	0.87	0.76 / 0.86
Text detector	free copy: bought	118,111	0.84	0.75 / 0.79
Text detector	free copy: free copy	2,376	0.83	0.75 / 0.77

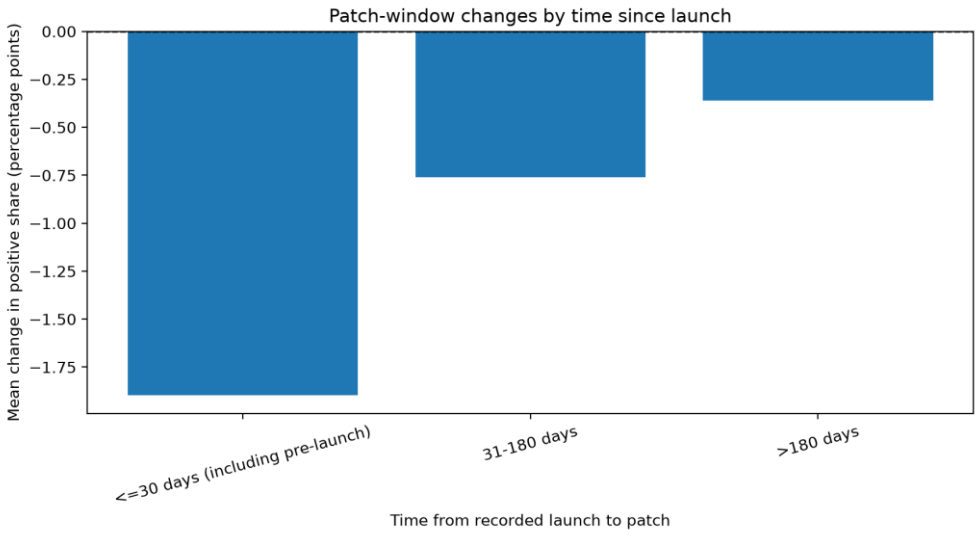


Figure 13. Patch impact by launch timing (change in positive-review share, points; Python/Matplotlib; notebooks/review2/patch_impact.ipynb).

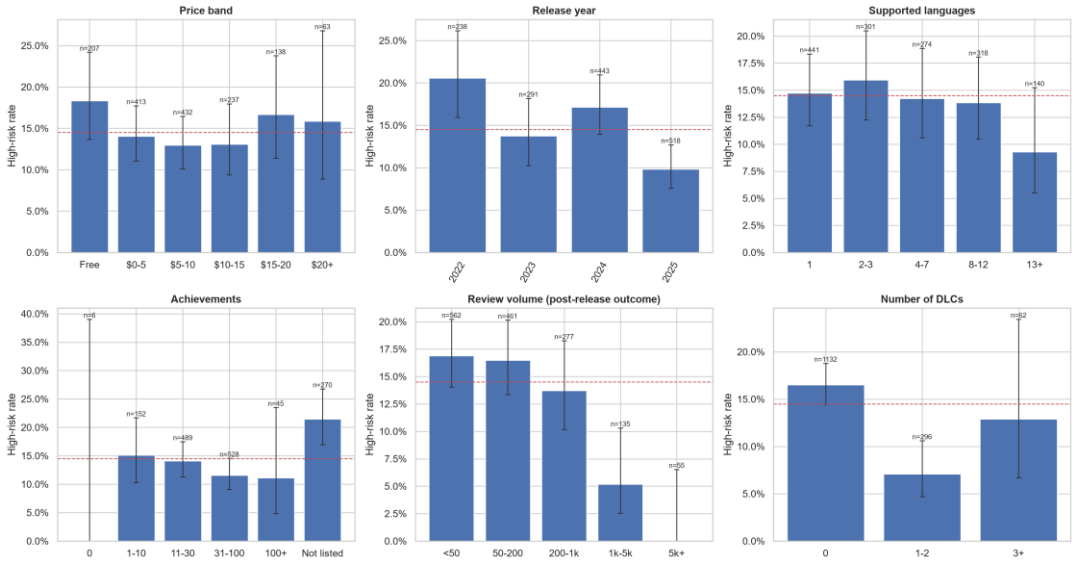


Figure 14. Share of Struggling games by price band, release year, languages, DLC, achievements and review volume with 95% intervals (Python/Seaborn; notebooks/02_EDA_Visualizations.ipynb).

Appendix C. Implementation details and dashboard user guide

Requirements: Python 3.12 and the pinned packages in `requirements.txt`. From the repository root:

- **Install:** `python -m venv .venv ; pip install -r requirements.txt`
- **Features and checks:** `python -m src.features`
- **Models:** `python -m src.models.review_models` (also `game_models`, `review_xgboost`, `game_xgboost`)
- **Model tables:** `python -m src.models.compare`
- **Tuning:** `python -m src.models.grid_search --task game --model xgboost`
- **Subgroup check:** `python -m src.models.subgroup_check`
- **Report:** `python -m src.report.build_capstone_report`

- **Dashboard:** `streamlit run src/dashboard/app.py`

User guide: *Market explorer*: filter by release year, price band, minimum reviews and tags; KPI cards and charts update. *Tag rules and complaints*: set stability, minimum lift and a tag or price to filter rules; open the complaint-themes and refund-window tabs. *Patch impact*: read the raw and matched-control results. *Launch-risk scorer*: describe a planned game (price, launch date, platforms, tags, categories) or load an existing game and press *Score this game*; read the predicted tier and scores (not probabilities).

Appendix D. Tracking-board evidence, contribution records and AI declaration

Project board: https://github.com/users/Shan713/projects/1 (68 cards, all Completed; card history, owners, due dates and evidence). Commit and pull-request history: https://github.com/Shan713/IndieGuard/pulls. The originality and AI-use declaration is signed by all members on the declaration page; the detailed AI disclosure is in Section 8.3.